

路线 1: QUBO / Ising 建模

QUBO / Ising 是七条路线的共同语言。你可以把它理解成：把一个选择题改写成能量函数，让“好答案”变成“低能量状态”。

这一步做不好，后面的退火、QAOA、constrained mixer 都会在错误地形上努力。

先看一个选择题

假设我们要在三个模型中选一个：

```
x_a = 1 表示选择 model_a  
x_b = 1 表示选择 model_b  
x_c = 1 表示选择 model_c
```

收益分别是 9、6、7。原问题是最大化收益：

$$\text{maximize } 9x_a + 6x_b + 7x_c$$

但 QUBO 通常写成最小化能量，所以先改成：

$$\text{minimize } -9x_a - 6x_b - 7x_c$$

如果只有这一步，最优解会把三个变量都设成 1，因为每选一个都能降低能量。可原问题要求“只能选一个”。于是加约束罚项。

Exactly-one 约束是：

$$x_a + x_b + x_c = 1$$

常见 penalty 是：

$$A \cdot (x_a + x_b + x_c - 1)^2$$

当刚好选一个时，括号里是 0，不罚。当一个都不选或选多个时，括号不为 0，就增加能量。

最终 QUBO 是：

$$\begin{aligned} &\text{minimize} \\ &-9x_a - 6x_b - 7x_c \\ &+ A \cdot (x_a + x_b + x_c - 1)^2 \end{aligned}$$

这就是 QUBO 的核心动作：目标函数保留偏好，约束变成罚项。

QUBO 标准形式

QUBO 的标准形式是：

$$\begin{aligned} \text{minimize } E(x) &= c + \sum_i q_i x_i + \sum_{i < j} q_{ij} x_i x_j \\ x_i &\in \{0, 1\} \end{aligned}$$

里面只有三类东西：

- 常数项 c
- 单变量项 $q_i x_i$
- 两变量交互项 $q_{ij} x_i x_j$

为什么只允许二次？因为很多退火器、Ising solver、QAOA cost Hamiltonian 都天然吃二次形式。超过二次的项通常要 quadratization，引入辅助变量降阶。

二次项的直觉

线性项表示单独选择某个变量的好坏。

$$-9x_a$$

代表 $x_a = 1$ 会让能量下降 9，因此倾向选择它。

二次项表示两个变量同时为 1 时的额外效果。

$$+20x_a x_b$$

代表同时选择 a 和 b 会被重罚。

$$-3x_c x_{boost}$$

代表 `model_c` 和 `boost` 同时出现时有协同收益。

所以 QUBO 不只是“选或不选”，它能表达组合效应。

约束为什么平方

等式约束通常长这样：

$$g(x) = 0$$

把它变成 penalty：

$$A * g(x)^2$$

平方有两个好处：

- 不管违反方向是正还是负，都会罚。
- 满足约束时 penalty 正好为 0。

对于 exactly-one:

$$g(x) = x_a + x_b + x_c - 1$$

如果选一个, $g(x) = 0$ 。如果选两个, $g(x) = 1$ 。如果三个都选, $g(x) = 2$, 罚得更重。

Ising 形式

Ising 变量不是 0/1, 而是 spin:

$$s_i \in \{-1, +1\}$$

Ising 能量写作:

$$H(s) = C + \sum_i h_i s_i + \sum_{i < j} J_{ij} s_i s_j$$

QUBO 和 Ising 可以互相转换。常用关系:

$$s_i = 2x_i - 1$$

$$x_i = (s_i + 1)/2$$

你可以把 QUBO 看成计算机科学味道的写法, 把 Ising 看成物理味道的写法。它们表达的是同一个能量地形。

变量编码

很多原问题变量不是 binary。

例如一个整数变量:

$$y \in \{0, 1, 2, 3, 4, 5, 6, 7\}$$

可以用 binary expansion:

$$y = b_0 + 2b_1 + 4b_2$$

也可以用 one-hot:

$$y = 0 \cdot z_0 + 1 \cdot z_1 + \dots + 7 \cdot z_7$$

$$z_0 + z_1 + \dots + z_7 = 1$$

binary expansion 省变量, 但约束和目标可能变复杂。one-hot 变量多, 但表达互斥和选择结构很直观。比赛中, 编码方式经常决定你能不能把模型压到可求规模。

高阶项二次化

如果目标里出现三变量乘积:

$$x_i x_j x_k$$

它不是 QUBO。常见处理是引入辅助变量：

$$y = x_i x_j$$

然后把高阶项改成：

$$y x_k$$

再加 penalty 保证 y 真的等于 $x_i x_j$ 。

直觉上，辅助变量是在帮你记住一个局部组合状态。但它会增加变量数和约束复杂度，不能滥用。

建模流程

一条健康的 QUBO 建模流程是：

1. 定义原问题变量
2. 选择 `binary` / `one-hot` / `domain-wall` / `integer encoding`
3. 写原始目标函数
4. 把 `maximization` 改成 `minimization`
5. 把硬约束变成 `penalty` 或交给其他机制
6. 展开成线性项和二次项
7. 检查系数范围、变量数、`coupler` 数
8. 小规模 `brute force` 验证
9. 解码回原问题变量
10. 重新计算原始目标和约束

第 10 步非常重要。不要把 QUBO energy 当成业务目标。QUBO energy 包含 penalty 和 offset，它只是求解器看到的地形。

比赛时能改哪些地方

变量设计

同一个问题可以有多种变量。路径问题可以用边变量，也可以用时间-位置变量。排产问题可以用开始时间变量，也可以用时间槽 `one-hot`。变量一变，整个 QUBO 难度都变。

编码方式

`binary`、`one-hot`、`domain-wall` 的变量数、耦合密度、可行性控制都不同。比赛时很容易靠编码改进拿到效果。

Penalty 权重

A 太小，非法解会赢。 A 太大，目标函数差异被淹没，求解器只看到约束墙。最好做 `penalty sweep`。

缩放

退火器和量子线路都对系数范围敏感。系数跨度太大，会让小差异不可见。

拆分

大问题不一定直接全量 QUBO。可以固定一部分变量，把剩下的不确定变量变成小 QUBO。

常见坑

把最大化忘记取负。

这会让算法找到最差解。

Penalty 没有压过目标收益。

例如违反约束能多拿 100 分，但 penalty 只有 20，求解器当然会违法。

只看 raw energy。

最低 QUBO energy 可能不可行。最后答案必须经过 decode、feasibility check、objective recomputation。

变量爆炸。

一个看似自然的 one-hot 编码可能让变量数从几十变成几千。要随时估算规模。

高阶项降阶过度。

每个辅助变量都要约束它的语义。辅助变量太多，模型会变硬。

和其他路线的关系

路线 2 接着处理约束质量。

路线 3 直接拿 QUBO/BQM 退火采样。

路线 4 把 QUBO/Ising 变成 cost Hamiltonian。

路线 5 会问：有些约束是否不要 penalty，而让 mixer 保持？

路线 6 会问：哪些变量不用全进 QUBO，可以经典求解或固定？

路线 7 会问：QUBO 图里哪些变量更重要，哪些变量能被学习策略 warm-start 或高置信度固定？

共读任务

用 `sample_problem.json` 手算一个简化 QUBO：

1. 只保留 `model_a`、`model_b`、`model_c`。
2. 写出收益项。
3. 加上 exactly-one penalty。
4. 比较 100、010、001、110 的能量。

如果你能解释为什么 110 会被罚，路线 1 就入门了。

延伸阅读

- `literature/requirements/01_qubo_ising_modeling_requirements.md`
- Glover, Kochenberger, Du, QUBO tutorial
- Lucas, Ising formulations of many NP problems